SPOTAK

Work & Study Spot Booking Platform

BIS351 – Business Information Systems Project

Student Name	Student ID
Mohammad Saad Aljuwair	2240006309
Yazid Hussain Alqahtani	22400004828
Abdulaziz Abdulrahman Aldossary	2240007946
Mohammad Ibrahim Alrasasi	2240004034

Table of Contents

1. Introduction

What is SPOTAK?

SPOTAK is a mobile-first web platform that helps users in Saudi Arabia's Eastern Province discover and reserve quiet cafes, co-working spaces, and study-friendly venues. The platform combines spot discovery, amenity filtering, detailed previews, and online reservation in a single lightweight interface that runs on any modern browser without requiring an installation or an account at a third-party service. The name SPOTAK plays on the words 'spot' and 'take' — the user takes (reserves) a spot — and the brand identity uses a calming teal color (#00A693) chosen to evoke focus and productivity, the two states the app helps users achieve.

Problem It Solves

Finding a suitable place to work or study outside the home is a recurring challenge for many residents of the Eastern Province. Cafes are often crowded during peak hours, Wi-Fi quality is unpredictable, and there is no consistent way to confirm whether a venue has power outlets, dedicated quiet hours, or available seating before traveling. Students who plan a group study session frequently arrive at a promising cafe only to discover it is fully booked or too noisy for productive work. Remote workers face the same problem when they need a reliable spot for a video call. SPOTAK addresses these gaps by surfacing verified amenity information for each spot and by allowing users to lock in a seat ahead of time, removing the uncertainty of an unplanned visit.

Target Users

- University students preparing for exams or working on long group projects who need a quiet, well-lit environment.
- Remote workers and freelancers searching for productive environments outside their home office.
- Professionals who need a presentable, quiet space for client meetings outside the corporate office.
- Casual visitors who simply want a nice cafe and prefer to confirm seating before driving across the city.

2. System Overview

Type of System

SPOTAK is a client-side web application built with vanilla HTML, CSS, and JavaScript. It deliberately avoids any frontend framework such as React, Vue, or Angular, which keeps the bundle size minimal (under one hundred kilobytes including images and external fonts) and ensures the codebase remains transparent and easy to read for an academic review. Because the app is purely static, it can be hosted on any web server or service that serves files, including GitHub Pages, Netlify, and Vercel, with zero configuration or build step.

Pages

- Home / Explore – the landing screen, used to browse and filter all available spots.
- Spot Detail – a deep dive into a single spot, showing photo, amenities, and a written description.
- Booking – a focused form for choosing a date, time, and group size to reserve a seat.
- Login – an authentication screen gating the booking flow behind a simple email + password check.
- Booking Confirmation – a success page that summarises the just-saved reservation.

Data Storage

All persistent data — the active user session, the currently selected spot, and the most recent booking — is stored in the browser's localStorage. This keeps the project fully client-side while still demonstrating realistic session and persistence behavior. Three namespaced keys are used: spotak_user for the logged-in session, spotak_current_spot for the spot the user is about to book, and spotak_last_booking for the most recent reservation displayed on the confirmation page. Using namespaced keys avoids collisions with other apps on the same domain and makes the stored data easy to inspect with the browser's developer tools.

3. Features Description

Search and Filter

Users can search for spots by name in real time through the top search bar. As the user types, the visible card list narrows down without any page reload. They can also filter the list by amenity using four chips at the top of the page: Quiet Focus, Fast Wi-Fi, 24/7 Open, and Meeting Room. Both filters are combinable, so a search for 'cafe' with the 'Quiet Focus' chip active will only display quiet cafes whose names contain the word 'cafe'.

Spot Details

Each spot has its own dedicated page showing a large hero photo, the location, three highlighted amenities (Wi-Fi speed, power-outlet availability, and ambient quietness) rendered as icon boxes, and a written description of the space. A back arrow in the top-left corner returns the user to the explore page while preserving any active filters.

Online Booking

The booking form collects a date, a time, and the number of people. The date input is pre-filled with today's date and uses the HTML5 min attribute to prevent selection of past dates at the browser level, with a JavaScript fallback for older browsers. A fee summary card shows the reservation cost (25 SAR) and the cancellation policy. The total is fixed at 25 SAR regardless of group size, matching the typical reservation-deposit model used by cafes.

Login and Session

Booking is gated behind a simple login form that validates a well-formed email address and a minimum password length. A valid session is stored in localStorage and recognized across pages, so the user stays signed in until they explicitly clear their browser data. Unauthenticated visitors who try to load the booking page are automatically redirected to the login screen.

Confirmation

After a successful booking, the user lands on a confirmation page with an animated SVG check icon and a summary of the booking that was just saved. The summary is rendered from localStorage, so it survives a page refresh — a small but important detail that mimics real-world e-commerce flows.

4. Technical Implementation

HTML

Five semantic HTML documents form the application: index.html, spot.html, booking.html, login.html, and confirmation.html. Each page uses semantic landmarks (main, header, section) and inline SVG icons rather than icon fonts to keep external dependencies at zero. Every page links to the same css/style.css and js/main.js, so global styling and behavior changes propagate instantly across the entire app.

CSS

A single stylesheet (css/style.css) drives the entire design system. CSS custom properties (variables) expose the brand color (#00A693), the background tone, the corner radius, and the elevation shadows. The layout is mobile-first with a maximum width of 430 pixels to mirror the phone-app feel of the original mockups. The 'DM Sans' font is loaded from Google Fonts at the top of the stylesheet.

JavaScript

All behavior lives in a single file: js/main.js. The script dispatches by page class on the DOMContentLoaded event and runs only the logic relevant to the current screen. Responsibilities include real-time search filtering, amenity-chip filtering, booking form validation (with a past-date guard), login persistence, and rendering the booking summary on the confirmation page. Each function carries a short comment block explaining its purpose so a future reader can navigate the file quickly.

File Structure

Markup, styles, scripts, and image assets are separated into dedicated folders (css/, js/, assets/images/). This makes the project easy to navigate and to extend with new pages or features later on. The root directory contains only the five HTML files plus the supporting documents (gantt.xlsx, report.docx, presentation.pptx).

Accessibility

Accessibility was treated as a first-class concern rather than an afterthought. Every form input is paired with an explicit <label> element wired through the for attribute, which means assistive technologies announce the field's purpose when the user moves focus into it. Error messages on the login and booking pages are rendered inside an aria-live="polite" region, so screen-reader users hear validation feedback the moment it appears without having to hunt for it on the page. The teal accent color (#00A693) was selected against the white background using the WCAG contrast checker to ensure a minimum AA-level ratio for normal text, and interactive elements maintain a visible focus outline so keyboard-only users can always see where they are. Inline SVG icons carry empty alt semantics through aria-hidden="true" because they are decorative — the text next to them already communicates the meaning. Tap targets on mobile follow the 44-pixel minimum recommended by Apple's Human Interface Guidelines.

Performance

Because SPOTAK ships no framework and no build step, the time-to-interactive on a first visit is dominated by the size of the HTML, CSS, and JavaScript files plus the external Google Fonts request. The total payload of code is under thirty kilobytes uncompressed and well under ten kilobytes after gzip, which means even on a slow 3G connection the app becomes interactive within a second. Images are loaded from Unsplash's CDN with size hints in the URL (w=1000) so the browser downloads an appropriately sized variant instead of a multi-megapixel original. The DOM stays small — fewer than two hundred nodes on the busiest page — which keeps style recalculations and reflows cheap as filters are applied in real time.

5. Code Walkthrough

This section walks through the most important pieces of code that power SPOTAK. The goal is not to reprint every line of the project, but to show the decisions behind each layer (CSS variables, page dispatch, search, validation, and persistence) so that someone unfamiliar with the codebase can understand how the parts fit together.

5.1 CSS Variables and Theming

At the top of style.css we declare a small set of CSS custom properties on the :root element. These act as a single source of truth for the visual identity of the entire app. Changing the value of --primary, for example, recolors every teal element across all five pages — buttons, icons, focus rings, brand text — without touching another line of CSS.

```
:root {
  --primary: #00A693;
  --bg: #F5F5F5;
  --white: #FFFFFF;
  --text: #1A1A1A;
  --gray: #888888;
  --radius: 16px;
}
```

Each rule in the stylesheet then references these variables instead of literal color values. For instance the fixed bottom button uses background-color: var(--primary), and the card backgrounds use var(--white) with var(--radius) for their rounded corners. This pattern is what professional design systems use under the hood, and it makes the stylesheet easy to maintain.

5.2 Page Dispatch

Rather than ship a separate script per page, all of SPOTAK's behavior lives in one main.js file. When the DOM is ready, the script inspects the class of the .app container and runs only the initializer relevant to the current page:

```
document.addEventListener('DOMContentLoaded', () => {
  const main = document.querySelector('.app');
  if (!main) return;

  if (main.classList.contains('spot-page')) {
    initSpotPage();
  } else if (main.classList.contains('booking-page')) {
    requireLogin();
    initBookingPage();
  } else if (main.classList.contains('login-page')) {
    initLoginPage();
  } else if (main.classList.contains('confirmation-page')) {
    initConfirmationPage();
  } else {
    initHomePage();
  }
```

```
});
```

This dispatch pattern keeps unrelated logic out of each page's execution path. The booking page also calls requireLogin() before its initializer, which redirects unauthenticated visitors to login.html. Centralizing the gate in one place means we cannot accidentally forget to protect a sensitive screen.

5.3 Real-Time Search and Filters

The explore page uses a single applyFilters function to handle both the search text and the amenity chips. Because each spot card stores its name in data-name and its amenity tags in data-tags, filtering reduces to a simple includes() check:

```
function applyFilters(searchText, amenityFilter) {
  const query = (searchText || '').trim().toLowerCase();
  const cards = document.querySelectorAll('.card');

  cards.forEach(card => {
    const name = (card.dataset.name || '').toLowerCase();
    const tags = (card.dataset.tags || '').toLowerCase();
    const matchesText = !query || name.includes(query);
    const matchesAmenity = !amenityFilter || tags.includes(amenityFilter);

    card.classList.toggle('hidden', !(matchesText && matchesAmenity));
  });
}
```

Hiding non-matching cards with a CSS class (rather than removing them from the DOM) means the search is instant and reversible: clearing the search box restores every card without any DOM reflow cost. The search input fires applyFilters on every 'input' event, which produces the real-time feel.

5.4 Booking Form Validation

The booking page combines native HTML5 validation (type=date, min, max, required) with a JavaScript layer that catches what HTML cannot — specifically, dates that are in the past. The submit handler runs each rule in sequence and writes the first failure into an aria-live region so screen readers announce the error:

```
form.addEventListener('submit', (e) => {
  e.preventDefault();
  errorBox.textContent = '';

  if (!date || !time || !people) {
    errorBox.textContent = 'Please fill out every field.';
    return;
  }

  const todayStart = new Date();
  todayStart.setHours(0, 0, 0, 0);
  const chosen = new Date(date + 'T' + time);
  if (chosen < todayStart) {
    errorBox.textContent = 'You cannot book a date in the past.';
```

```
    return;
  }

  // ... persist booking and redirect
});
```

Notice how each guard short-circuits with an early return: this avoids deeply nested if-else chains and makes each rule independently easy to read. The final step serializes the booking to JSON and writes it to localStorage under spotak_last_booking, then redirects to the confirmation page.

5.5 Persistence with localStorage

Because SPOTAK has no backend, every piece of state that needs to survive a page navigation is written to localStorage. The pattern is consistent across the app: serialize an object with JSON.stringify on the way in, parse it back with JSON.parse on the way out, and use namespaced keys to avoid collisions:

```
// Save the booking when the form is submitted
const booking = { spotName, date, time, people, fee: 25 };
localStorage.setItem('spotak_last_booking', JSON.stringify(booking));

// Read the booking back on the confirmation page
const raw = localStorage.getItem('spotak_last_booking');
if (raw) {
  const b = JSON.parse(raw);
  document.getElementById('sumDate').textContent = formatDate(b.date);
}
```

The same approach is used for the login session (spotak_user) and the currently selected spot (spotak_current_spot). Wrapping every JSON.parse in a try/catch on the confirmation page protects against the rare case where storage contains malformed data, so a corrupted entry never crashes the UI.

5.6 Animations and Polish

The animated checkmark on the confirmation page is a pure SVG + CSS effect. Two <path> elements (a circle and a check) have their stroke-dasharray set equal to their length, so the stroke is initially hidden. A CSS keyframe animation then reduces stroke-dashoffset to zero, producing the drawing effect:

```
@keyframes drawCircle { to { stroke-dashoffset: 0; } }
@keyframes drawCheck  { to { stroke-dashoffset: 0; } }
```

The spot cards on the home page fade upward with a staggered delay using the same animation pattern, which gives the explore screen a polished feel on first load without any JavaScript at all.

6. Project Planning (Gantt Chart)

A Gantt chart was prepared to plan and track the build of SPOTAK. The chart covers the work window from May 6 to May 14, 2026 and uses a teal bar to indicate the days each task is active. The full schedule is reproduced below:

Task	Start	End	May 06	May 07	May 08	May 09	May 10	May 11	May 12	May 13	May 14
Requirements Analysis	May 06	May 07									
UI Design & Mockups	May 07	May 08									
HTML Structure	May 08	May 09									
CSS Styling	May 09	May 10									
JavaScript Logic	May 10	May 12									
Testing & Debugging	May 12	May 13									
Final Review & Deployment	May 13	May 14									

Overlap between adjacent tasks is intentional. Testing began while final JavaScript polish was still in progress, and CSS styling started a day before HTML was fully complete. This kept the schedule realistic and avoided idle days, which is a common pattern in small software projects where one developer wears every hat.

7. Challenges & Solutions

Responsive Mobile-First Layout

Challenge: the original mockups were designed for a phone screen, but the app needed to display correctly on laptops as well. Building a layout that looked like a phone app on desktop without breaking on actual phones required careful CSS work. A naive approach would force horizontal scrolling on mobile or stretch the cards to uncomfortably wide proportions on a 27-inch monitor.

Solution: the .app container was given max-width: 430px and centered with margin: 0 auto. This produces a faithful phone-app frame on wide screens while collapsing to the viewport width on small devices, with no extra media queries needed. The fixed bottom CTA button uses the same max-width plus a translateX trick to stay centered relative to the phone frame rather than the full window.

Persistence Without a Backend

Challenge: the project requirements forbid a server, but real bookings still had to be saved and re-read on the confirmation page. Storing structured objects in a way that survived page navigation needed a deliberate approach, and corrupted data could not be allowed to crash the UI.

Solution: bookings, the active session, and the currently viewed spot are all serialized with JSON.stringify and written to localStorage under namespaced keys. On the next page they are parsed back into objects and rendered. Every JSON.parse call is wrapped in a try/catch on the confirmation page so a corrupted entry produces an empty summary instead of a broken page.

Form Validation Logic

Challenge: the booking form had to reject empty fields, dates in the past, and group sizes outside a reasonable range, all without using a third-party library. Validation also had to be accessible — screen-reader users had to hear the error message immediately, not on a page reload.

Solution: the submit handler runs each validation rule in sequence and writes the first failure into an aria-live error region beneath the form. Native HTML5 attributes (type, min, max, required) provide the first layer of defense, while the JavaScript layer takes care of checks that HTML alone cannot enforce, such as rejecting a chosen datetime that falls in the past.

Cross-Browser Consistency

Challenge: the app had to look and behave identically across Chrome, Safari, Firefox, and Edge on both desktop and mobile, but each browser ships subtle differences in how it renders form controls, date pickers, and CSS gradients. Safari on iOS in particular zooms the page when a user taps an input whose font size is below sixteen pixels, which would break the layout. Older versions of Firefox render

the native date picker very differently from Chrome, which could confuse a user who expects the same calendar widget everywhere.

Solution: all input fields were given a minimum font size of sixteen pixels to disable iOS auto-zoom, and a small set of CSS resets normalizes padding, border radius, and appearance across browsers. The date picker is kept as the native control rather than swapping it for a custom widget — this trades a uniform look for guaranteed reliability, since the native picker is already accessible, localized, and battle-tested. A short manual test pass on each target browser was added to the deployment checklist before the final review day on the Gantt schedule, which caught two minor rendering bugs that would otherwise have shipped.

8. Conclusion

Summary

SPOTAK demonstrates that a full booking experience — discovery, filtering, detail view, reservation, and confirmation — can be delivered as a small, framework-free web app. The five pages share a single stylesheet and a single script, and the entire project remains under a few hundred kilobytes including external fonts. Despite its small size, the app implements every flow a real reservation product would need: search, filter, detail view, gated booking, validation, persistence, and a confirmation summary.

What Was Learned

- How to structure a multi-page web app with shared CSS and JavaScript instead of duplicating code per page.
- How to design a phone-style interface that adapts gracefully to larger screens with a single max-width rule.
- How localStorage can replace a backend for academic and demo projects while still demonstrating realistic persistence.
- How to plan a one-week build with a Gantt chart and keep tasks overlapping so no day is wasted.
- How CSS custom properties simplify theming and make a design system maintainable.

Future Improvements

- Replace localStorage with a real backend (Node.js + PostgreSQL or Firebase) so bookings persist across devices and users.
- Integrate a payment gateway (e.g., Mada, STC Pay, or Stripe) for the reservation fee instead of a placeholder.
- Add a map view using Google Maps or Mapbox that plots each spot on a real map of the Eastern Province.
- Allow venue owners to manage their own listings through an admin dashboard with photo uploads and availability calendars.
- Add user reviews and ratings so the community can build trust around each listed spot.
- Build a native mobile app wrapper using Capacitor or React Native to publish SPOTAK to the App Store and Google Play.